01 QUESTION ONE · DBMS

What do you mean by DBMS? Explain the purpose of DBMS.

4 A4 PAGES
SQL PROGRAM ON PAGE 4

WHAT IS A DBMS?

A **DBMS (Database Management System)** is software that is used to create a database, store data in it, manage that data, and retrieve it correctly whenever it is needed. The job of a DBMS is not only to hold data, but also to organise it, to keep it secure, and to allow several users to work with it at the same time. We can understand it with the help of a librarian: just as a librarian keeps the books in a proper order and hands over whichever book is asked for, a DBMS keeps data in a proper order and supplies the correct data when required.

Without a DBMS, data would be kept in ordinary flat files — for example, in a text file or a spreadsheet — and this creates many problems. The same information has to be written again and again in different files (redundancy), a change made in one place stays wrong in another (inconsistency), the data is not protected from unauthorised users, and finding a single record inside many files becomes slow and difficult. A DBMS removes all of these problems by keeping the data in one organised place under one control.

Examples of DBMS

- **Oracle** — the most powerful and widely used professional DBMS, common in banks and large organisations.
- **MySQL** — free and open source, used by many websites.
- **Microsoft SQL Server** — Microsoft's DBMS, popular in the corporate world.
- **PostgreSQL** — an open-source DBMS with advanced features.
- **Microsoft Access** — a simple DBMS used for small projects and learning.

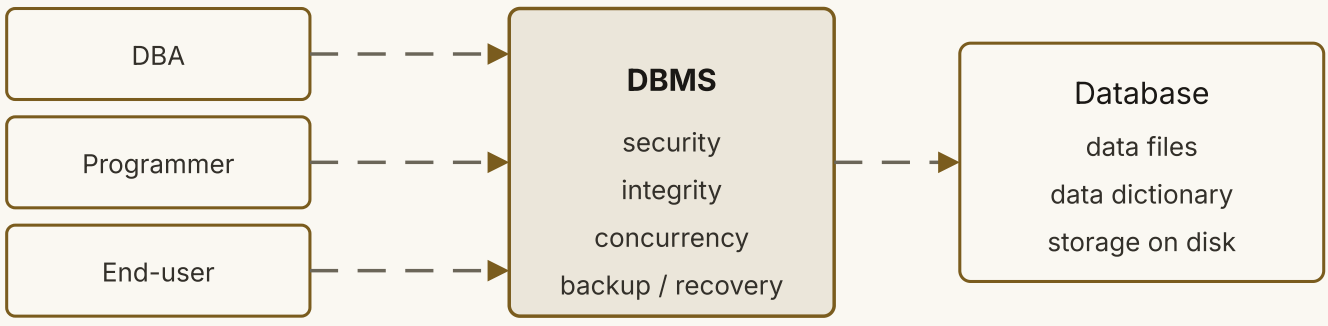


FIG. 1 — A DBMS DOES NOT SIT BESIDE THE DATA; IT STANDS **BETWEEN** THE USERS AND THE DATA, CHECKING EVERY REQUEST BEFORE IT IS ALLOWED TO TOUCH THE RECORDS.

PURPOSE OF DBMS

A DBMS is needed because as soon as data grows, managing it by hand becomes impossible. It solves all the problems of file-based storage and keeps the data organised, safe and easy to reach. Its main purposes are described below.

1. **To reduce Data Redundancy:** the data is stored in one place only, so the same information does not have to be written again and again. This saves storage space and keeps the data consistent.
2. **Data Sharing:** several users can use the same data at the same time. In a bank, for example, a teller can view a customer's account at the very moment a manager is preparing a report.
3. **Data Integrity:** the DBMS applies rules (constraints) such as primary keys, data types and NOT NULL, so the data always stays correct and valid.
4. **Data Security:** the DBMS provides user names and permissions, so unauthorised people can neither view nor change the data.
5. **Data Consistency:** if data is changed in one place, it is updated everywhere automatically, so a wrong value never appears anywhere.
6. **Efficient Data Access:** with the help of indexing and query optimisation, information is found quickly even inside a very large amount of data.
7. **Backup and Recovery:** the DBMS takes regular backups, and if data is lost or the system fails, the data can be restored.
8. **Reduced Application Time:** because the DBMS already handles storage, security and concurrency, the programmer needs to write much less code for each new application.

It should be clear from this list that a DBMS is not simply a place in which data is kept. It is an active manager that stands between the data and the people who use it. Every time a form is filled, a report is printed or a payment is made, the DBMS is quietly checking the rules, granting or refusing access, keeping the records consistent and recording the change so that it can be undone if something goes wrong. This is why a modern organization cannot be imagined without a DBMS, and why the subject is the foundation of every course on information systems.

COMPONENTS OF A DBMS

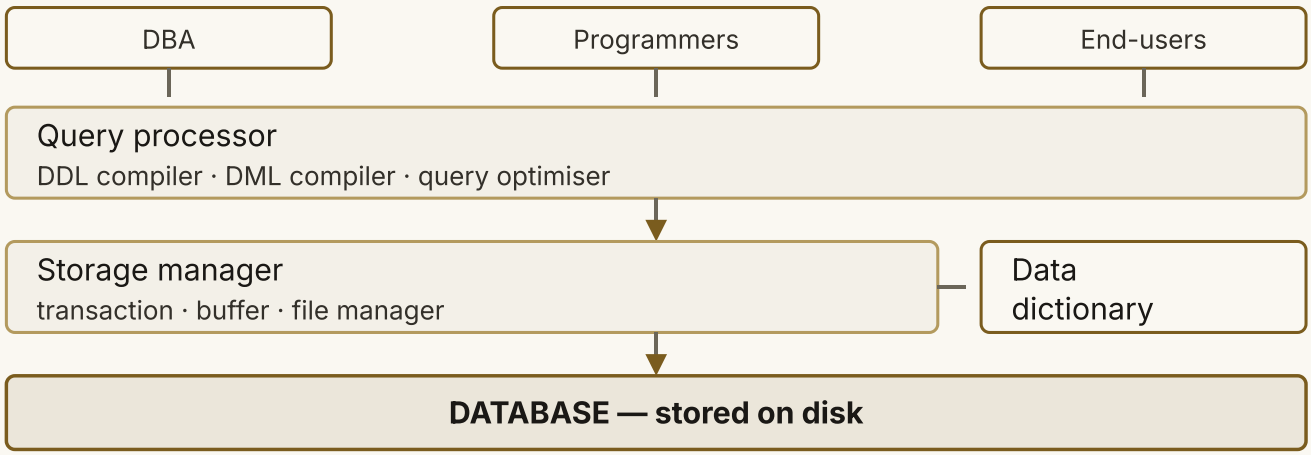
A DBMS works with the help of five important components. **Hardware** is the physical computers and storage devices on which the data is kept. **Software** is the DBMS program itself, which stands between the users and the database. **Data** is the actual information stored in the database. **Procedures** are the rules and instructions that tell how the DBMS is to be used and managed. **Users** are the people who work with the system — the database administrator, the application programmers and the ordinary end-users.

An important part inside a DBMS is the **Data Dictionary**, also called the system catalog. It stores **metadata** — that is, data about data. It keeps a record of every table name, column name, data type, constraint and user permission. Whenever a query is run, the DBMS first looks in the data dictionary to understand the shape of the table, and only then does it fetch the data. Without the data dictionary the DBMS would not know which tables exist at all.

All of these parts together make a DBMS a complete system rather than a mere store of files — which is why it is used in every field, from a college's student records to a bank's transactions.

USERS OF A DBMS

A DBMS is used by three kinds of people, each needing a different level of access. The **Database Administrator (DBA)** is the most powerful: he creates the database, decides its structure, grants permissions, takes backups and repairs the system if it fails. The **application programmers** write the programs that ordinary users work with, and are given only the access their program needs. The **end-users** finally use those programs — a clerk entering a form, a teller checking a balance — and never touch the database directly; they see only the screen in front of them. This separation of roles is one of the greatest strengths of a DBMS, because it keeps sensitive data away from careless hands.



EVERY REQUEST IS CHECKED BEFORE IT IS ALLOWED TO TOUCH THE DATA

FIG. 2 — THE LAYERS OF A DBMS. USERS SEND A REQUEST TO THE QUERY PROCESSOR, THE QUERY PROCESSOR ASKS THE STORAGE MANAGER FOR THE BYTES, AND THE DATA DICTIONARY IS CONSULTED AT EVERY STEP SO THAT THE SYSTEM ALWAYS KNOWS THE SHAPE OF ITS OWN TABLES.

SQL PROGRAM (ORACLE)

THE PROGRAM BELOW CREATES A STUDENT TABLE, INSERTS RECORDS AND RETRIEVES THEM — THE BASIC WORK OF A DBMS. COPY IT ON PAGE 4.

DBMS_DEMO.SQL

```
-- dbms_demo.sql : create, insert and retrieve data
CREATE TABLE student (
    roll_no NUMBER(5) PRIMARY KEY,
    name    VARCHAR2(40) NOT NULL,
    course  VARCHAR2(20),
    marks   NUMBER(5,2)
);
INSERT INTO student VALUES (101, 'Rahul Kumar', 'BCA', 78.50);
INSERT INTO student VALUES (102, 'Anjali Singh','BCA', 85.25);
INSERT INTO student VALUES (103, 'Vikash Yadav','BBA', 66.00);

SELECT name, course, marks
FROM   student
WHERE  course = 'BCA'
ORDER  BY marks DESC;
```

Output of the Query

OUTPUT

```
NAME            COURSE    MARKS
--------------- --------- -----
Anjali Singh    BCA        85.25
Rahul Kumar     BCA        78.50
```

In the output above the DBMS displayed only the records whose course was “BCA” and arranged them according to marks. This is exactly the work of a DBMS — to organise data and to supply only what is asked for.

CONCLUSION

A DBMS is software that stores, manages and retrieves data efficiently. Its purpose is to reduce data redundancy, to provide sharing and security, and to keep the data consistent and always available. Without a DBMS, large organisations could not manage their data, which is why it is used everywhere today.

02 QUESTION TWO · NORMALIZATION

Explain Normalization with Example.

4 A4 PAGES
SQL EXAMPLE ON PAGE 4

WHAT IS NORMALIZATION?

Normalization is the process of designing a database in which large tables are divided into smaller, related tables so that data redundancy (the same data being stored again and again) is reduced and data integrity (correctness of data) is improved. The aim is that every piece of data should be stored in **one place only** and should be linked to the other tables by keys. This makes the database organised and removes many errors.

Normalization is needed because a badly designed database suffers from three types of problem, called **anomalies**. An **Insertion Anomaly** occurs when new data cannot be inserted without also writing unrelated old data. An **Update Anomaly** occurs when the same fact is stored in many rows, so a change has to be made in every one of them and any miss leaves wrong data behind. A **Deletion Anomaly** occurs when deleting one record accidentally removes some other necessary information along with it. Normalization removes all three.

Benefits of Normalization

- **Less Data Redundancy:** the same data is not stored repeatedly, so storage is saved.
- **Better Data Integrity:** the data stays consistent and a wrong value does not appear anywhere.
- **Anomalies Removed:** the insertion, update and deletion anomalies disappear.
- **Organised Design:** the tables become small and related, so the database is easier to understand and maintain.
- **Easier Queries:** since the tables are clean, joins and reports give correct results.

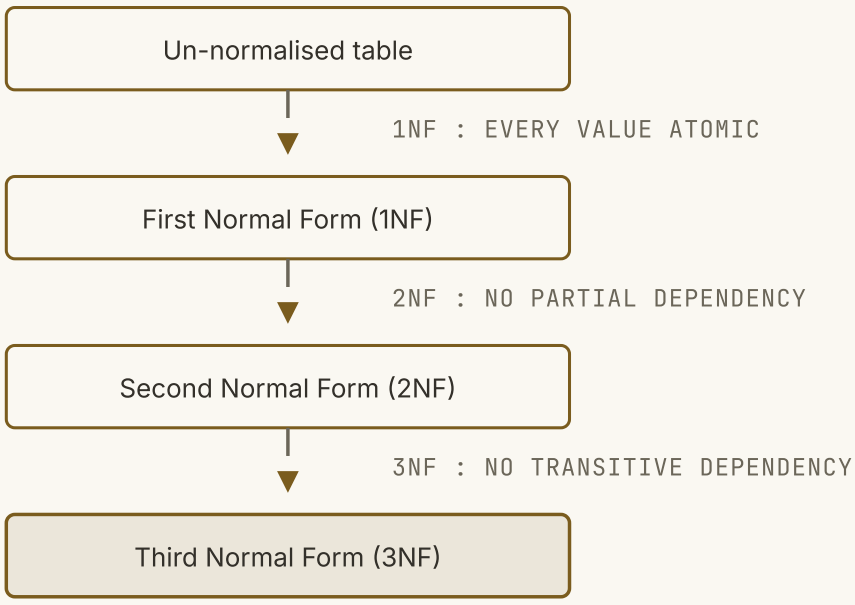


FIG. 1 — THE NORMALIZATION LADDER. EACH STEP REMOVES ONE KIND OF DEPENDENCY, SO THE TABLE BECOMES SMALLER, CLEANER AND FREE OF ANOMALIES.

FIRST NORMAL FORM (1NF)

A table is in **1NF** when every column contains an **atomic (single) value** — that is, each cell holds one value only, and no field contains a list or a repeating group. Every row must also be unique. This is the very first step of normalization.

Suppose we have an un-normalised table in which several courses of one student are written together in the same cell. This is a repeating group and is not allowed:

ROLL_NO	NAME	COURSES	CITY
101	Rahul	BCA, BBA	Patna
102	Anjali	BCA	Patna

To bring it into 1NF we split the multi-valued *Courses* column into separate rows, so that every cell holds only one value:

ROLL_NO	NAME	COURSE	CITY
101	Rahul	BCA	Patna
101	Rahul	BBA	Patna
102	Anjali	BCA	Patna

Now every cell contains a single atomic value and the repeating group is gone, so the table is in 1NF.

It is worth noting why 1NF alone is not enough. Making every cell atomic removes the repeating groups, but it does not yet remove the **redundancy** inside the rows. In the table above, the name “Rahul” and the city “Patna” are both written twice, once for each course, and this repetition is itself a source of danger: if Rahul changes his city, two rows must be changed, and missing one of them would leave the data inconsistent. The next two normal forms are therefore introduced to remove exactly this kind of duplication, and they are discussed on the next page.

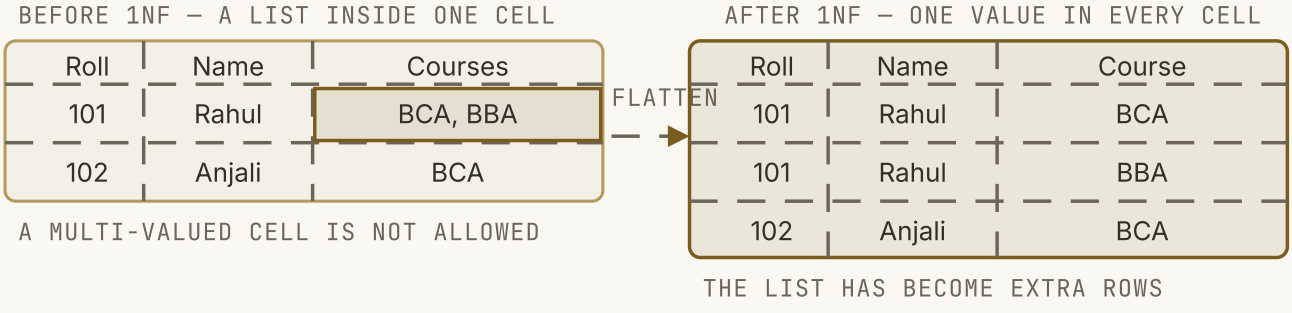


FIG. 2 — REACHING FIRST NORMAL FORM. THE “COURSES” CELL THAT HELD TWO VALUES IS BROKEN OPEN, AND EACH VALUE IS GIVEN A ROW OF ITS OWN. NOTHING HAS BEEN ADDED AND NOTHING HAS BEEN LOST — THE SAME FACTS ARE SIMPLY STORED ONE VALUE AT A TIME.

SECOND NORMAL FORM (2NF)

A table is in **2NF** when it is already in 1NF and there is **no partial dependency** in it. A partial dependency occurs when a non-key column depends on only **part** of the primary key. This can happen only when the primary key is **composite** — that is, made of two or more columns.

Suppose we have the table `ENROLLMENT (Roll_No, Course_Code, Student_Name, Course_Name)` whose primary key is the pair `(Roll_No, Course_Code)`. Here `Student_Name` depends only on `Roll_No` and `Course_Name` depends only on `Course_Code`; both are partial dependencies.

To reach 2NF we split the table into two — **STUDENT (Roll_No, Student_Name)** and **ENROLLMENT (Roll_No, Course_Code, Course_Name)**. The partial dependencies are now gone.

THIRD NORMAL FORM (3NF)

A table is in **3NF** when it is already in 2NF and there is **no transitive dependency** in it. A transitive dependency occurs when a non-key column depends not on the primary key but on **another non-key column**.

Suppose the table is `STUDENT (Roll_No, Student_Name, Dept_Id, Dept_Name)` whose primary key is `Roll_No`. Here `Dept_Name` does not depend directly on `Roll_No`; it depends on `Dept_Id` — that is, `Roll_No → Dept_Id → Dept_Name`. This is a transitive dependency. To reach 3NF we move the department details into a separate table — **STUDENT (Roll_No, Student_Name, Dept_Id)** and **DEPARTMENT (Dept_Id, Dept_Name)**. The transitive dependency is now gone.

Thus 1NF removes repeating groups, 2NF removes partial dependencies and 3NF removes transitive dependencies.

A useful way to remember the three forms is by what each one forbids. 1NF forbids a **list inside a cell**; every value must stand alone. 2NF forbids a **part-key dependency**; no non-key column may depend on only one half of a composite key. 3NF forbids a **non-key dependency**; no non-key column may depend on another non-key column. Beyond these stands the **BCNF (Boyce-Codd Normal Form)**, a slightly stricter form of 3NF that is used in advanced design. In ordinary college work, a table that is in 3NF is considered properly normalised, and this is why the three forms are studied together as the standard path of database design.

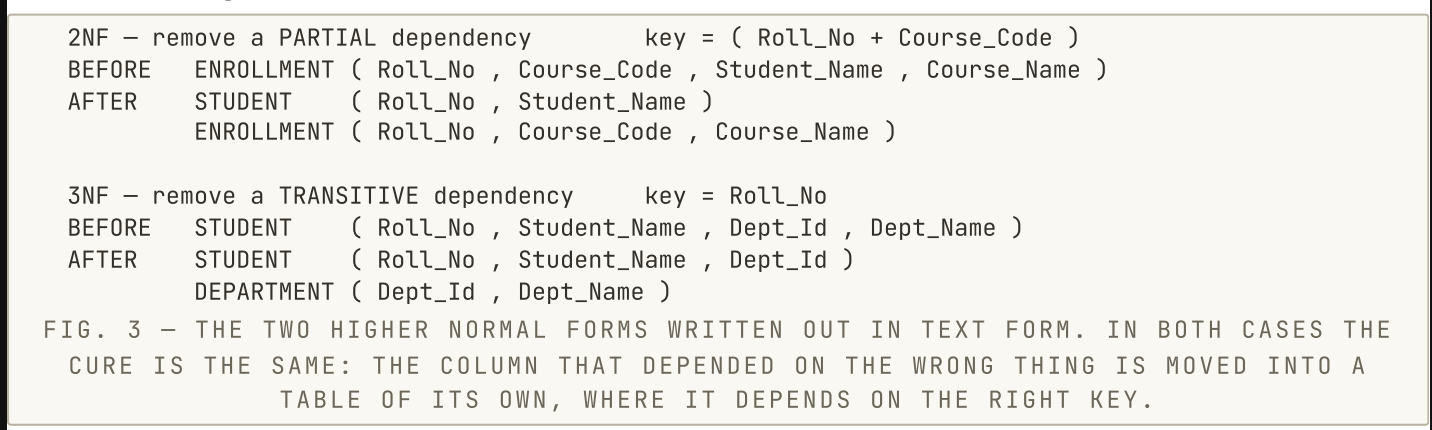


FIG. 3 — THE TWO HIGHER NORMAL FORMS WRITTEN OUT IN TEXT FORM. IN BOTH CASES THE CURE IS THE SAME: THE COLUMN THAT DEPENDED ON THE WRONG THING IS MOVED INTO A TABLE OF ITS OWN, WHERE IT DEPENDS ON THE RIGHT KEY.

SQL PROGRAM (ORACLE)

NORMALISATION CARRIED OUT STEP BY STEP FROM 1NF TO 3NF. COPY THIS PROGRAM ON PAGE 4.

NORMALIZATION.SQL

```sql
-- 1NF : one atomic value in every cell
CREATE TABLE student_1nf (
    roll_no NUMBER(5),
    name    VARCHAR2(40),
    course  VARCHAR2(20),
    city    VARCHAR2(30)
);
INSERT INTO student_1nf VALUES (101,'Rahul','BCA','Patna');
INSERT INTO student_1nf VALUES (101,'Rahul','BBA','Patna');
INSERT INTO student_1nf VALUES (102,'Anjali','BCA','Patna');

-- 2NF : composite key split, partial dependency removed
CREATE TABLE enrollment_2nf (
    roll_no     NUMBER(5),
    course_code VARCHAR2(10),
    course_name VARCHAR2(30),
    PRIMARY KEY (roll_no, course_code)
);

-- 3NF : transitive dependency moved to a new table
CREATE TABLE department (
    dept_id   NUMBER(4) PRIMARY KEY,
    dept_name VARCHAR2(30)
);
CREATE TABLE student_3nf (
    roll_no NUMBER(5) PRIMARY KEY,
    name    VARCHAR2(40),
    dept_id NUMBER(4) REFERENCES department(dept_id)
);
INSERT INTO department VALUES (10, 'Computer Science');
INSERT INTO student_3nf VALUES (101, 'Rahul', 10);

-- join the normalised tables to view the data
SELECT s.name, d.dept_name
FROM   student_3nf s JOIN department d ON s.dept_id = d.dept_id;
```

Output of the Query

OUTPUT

```
NAME       DEPT_NAME
---------- ------------------
Rahul      Computer Science
```

CONCLUSION

Normalization makes a database organised. In 1NF repeating groups are removed, in 2NF partial dependencies are removed and in 3NF transitive dependencies are removed. Step by step the redundancy falls and the integrity of the data rises, which is exactly why every good database is normalised.

03 QUESTION THREE · KEYS

Explain Primary Key, Foreign Key & Unique Key with Example.

4 A4 PAGES
SQL EXAMPLE ON PAGE 4

WHAT IS A KEY?

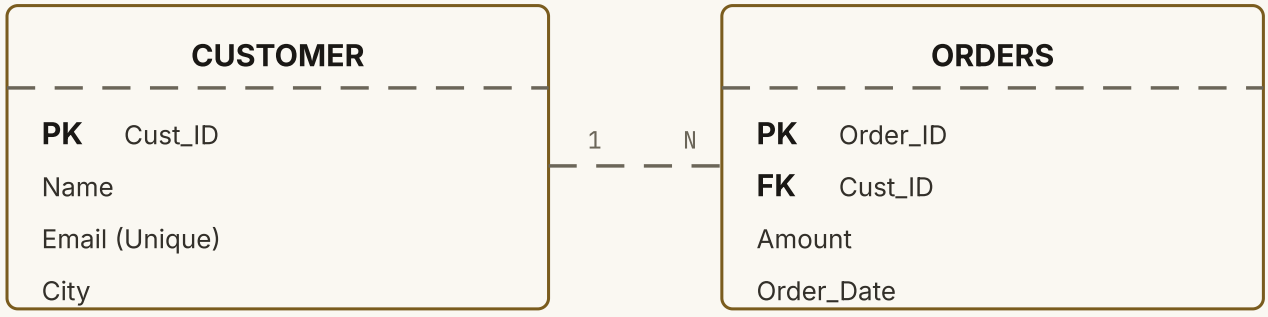
In a database table it is essential to identify every record uniquely, so that one record can be told apart from all the others. **Keys** are the columns used for exactly this purpose. A key is a column, or a group of columns, that makes a record different from every other record and also establishes a relationship between tables. The three most important keys are the **Primary Key, the Foreign Key and the Unique Key**.

1. PRIMARY KEY

A **Primary Key** is a column (or a group of columns) that uniquely identifies every record of a table. It has two strict rules — it must contain **unique values** (no two values may be the same) and it **cannot contain a NULL** (a blank value). A table can have only **one** primary key. Its job is to identify records uniquely and to keep every row different from all the others.

For example, in a student table `Roll_No` can be the primary key, because every student's roll number is different and two students cannot share the same one. In the same way, in a bank account table `Account_No` is the primary key, in an Aadhaar record the `Aadhaar_Number` is the primary key, and in an order table the `Order_ID` is the primary key. A primary key may be a single column or a combination of columns (a composite key).

The primary key is the most important of all the keys because everything else in the design leans upon it. When a foreign key in another table points to this table, it points at the primary key, and the DBMS checks the value against it every time a row is inserted. This is why a primary key must never be changed casually: if it is changed, every foreign key that refers to it must be changed as well, or the relationship will break. Good designers therefore choose, as the primary key, a value that will never need to change — usually a simple number such as a roll number, an account number or an order number — and never a value such as a person's name or address, which may well change during their lifetime.



FK CUST_ID REFERS TO CUSTOMER(CUST_ID) - THIS IS REFERENTIAL INTEGRITY

FIG. 1 — THE THREE KEYS WORKING TOGETHER. **CUST_ID** IS THE PRIMARY KEY OF CUSTOMER AND A FOREIGN KEY IN ORDERS, WHILE **EMAIL** IS A UNIQUE KEY.

2. FOREIGN KEY

A **Foreign Key** is a column in one table that refers to the **primary key of another table**. Its main job is to create a **relationship** between two tables and to enforce **referential integrity** — which means that a value may exist in one table only if it already exists in the other. A foreign key therefore prevents a wrong or "orphan" link from being formed, and keeps the data consistent across the tables.

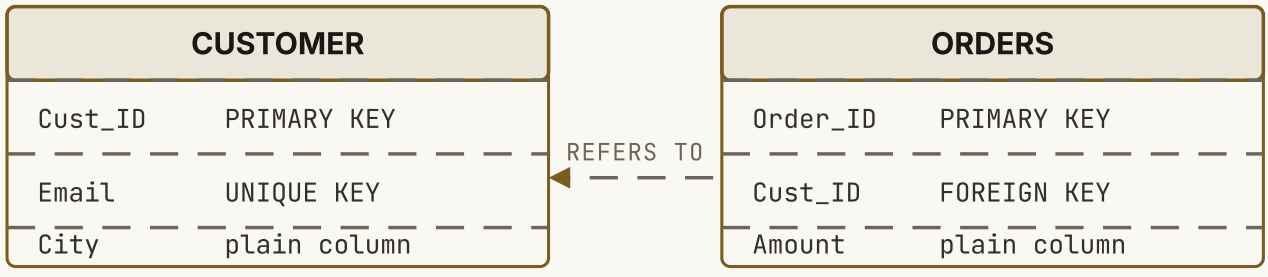
For example, suppose there is an `ORDER` table for customer orders and a `CUSTOMER` table for customer details. The ORDER table will have a column `Customer_ID` that refers to the `Customer_ID` (the primary key) of the CUSTOMER table. This makes sure that every order belongs to a real, existing customer — an order from an unknown customer simply cannot be entered. Unlike a primary key, a foreign key **may contain NULL** and a table **may have more than one** foreign key.

3. UNIQUE KEY

A **Unique Key** makes sure that **all the values in a column are different** — no two may be the same. The two differences from a primary key are that a table may have **more than one** unique key and that a unique key **may allow one NULL value**. A unique key is used when a column must be free of duplicates but need not be the table's identity.

For example, in a user table the `Email` column may be made a unique key, because every user's email must be different; in the same way a mobile number (with the country code) is often made a unique key. This stops two users from signing up with the same email address.

It is helpful to keep the three keys apart in one line each. A **primary key** says *"this row is this row, and nothing else"* — it gives identity. A **foreign key** says *"this row belongs to that row"* — it gives relationship. A **unique key** says *"no two rows may share this value"* — it gives prevention of duplicates. A single column may even play more than one of these roles: a column that is a foreign key may also be declared unique, so that only one order may reference a given reference number. What never changes is the rule that a table has exactly one primary key, for a table can have only one identity, though it may have many relationships and many unique columns.



AN ORDER WHOSE CUST_ID DOES NOT EXIST IN CUSTOMER IS REFUSED

FIG. 2 — REFERENTIAL INTEGRITY IN ONE PICTURE. THE ORDERS TABLE MAY POINT AT A CUSTOMER ONLY IF THAT CUSTOMER REALLY EXISTS; THE CUSTOMER TABLE PROTECTS ITS OWN IDENTITY WITH A PRIMARY KEY AND ITS DUPLICATES WITH A UNIQUE KEY.

COMPARISON OF THE THREE KEYS

BASIS	PRIMARY KEY	FOREIGN KEY	UNIQUE KEY
Definition	Identifies a record uniquely	Links two tables together	Keeps all values different
NULL allowed?	No	Yes	One NULL allowed
Uniqueness	All values unique	Need not be unique	All values unique
How many per table	Only one	More than one possible	More than one possible
Purpose	Identity of a row	Referential integrity	Prevent duplicates
Example	Roll_No in Student	Customer_ID in Order	Email in User

EXAMPLE TABLE DESIGN

To see the three keys working together, take the two tables **CUSTOMER** and **ORDERS**. In the CUSTOMER table, `Cust_ID` is the **primary key** and `Email` is a **unique key**. In the ORDERS table, `Order_ID` is the **primary key**, and `Cust_ID` is a **foreign key** that points back to the primary key of the CUSTOMER table.

Because of the foreign key, an order can never be entered for a customer who does not exist. Because of the unique key, two customers can never share an email. And because of the primary keys, every customer and every order can be identified without any confusion. The three keys together keep the database both correct and connected.

In a real project, the keys are usually planned before a single table is created. The designer first decides what each table represents — a customer, an order, a course, a student — then chooses a primary key for each table, and finally draws the relationships between the tables by adding foreign keys. Only after this map is ready does the actual work of writing the CREATE TABLE statements begin. A database designed in this order stays correct for years, while a database thrown together table by table soon fills with duplicated and contradictory data. This is why keys are studied so early in every database course: they are not a detail of the design but its foundation.

SQL PROGRAM (ORACLE)

PRIMARY, FOREIGN AND UNIQUE KEYS USED TOGETHER IN TWO RELATED TABLES. COPY THIS PROGRAM ON PAGE 4.

KEYS_DEMO.SQL

```sql
-- CUSTOMER : primary key + unique key
CREATE TABLE customer (
    cust_id   NUMBER(6) PRIMARY KEY,          -- PRIMARY KEY
    cust_name VARCHAR2(40) NOT NULL,
    email     VARCHAR2(60) UNIQUE             -- UNIQUE KEY
);
-- ORDERS : primary key + foreign key
CREATE TABLE orders (
    order_id NUMBER(6) PRIMARY KEY,           -- PRIMARY KEY
    cust_id  NUMBER(6) REFERENCES customer(cust_id),  -- FOREIGN KEY
    amount   NUMBER(8,2)
);
INSERT INTO customer VALUES (1, 'Rahul Kumar',  'rahul@mail.com');
INSERT INTO customer VALUES (2, 'Anjali Singh', 'anjali@mail.com');

INSERT INTO orders VALUES (101, 1, 2500.00);
INSERT INTO orders VALUES (102, 2, 1800.00);
INSERT INTO orders VALUES (103, 1, 3200.00);

-- the foreign key joins the two tables
SELECT o.order_id, c.cust_name, c.email, o.amount
FROM   orders o JOIN customer c ON o.cust_id = c.cust_id;
```

Output of the Query

OUTPUT

```
ORDER_ID CUST_NAME      EMAIL             AMOUNT
-------- -------------- ----------------- ------
101      Rahul Kumar    rahul@mail.com      2500
102      Anjali Singh   anjali@mail.com     1800
103      Rahul Kumar    rahul@mail.com      3200
```

CONCLUSION

Keys are essential to a database. The Primary Key identifies a record uniquely, the Foreign Key creates a relationship between two tables and enforces referential integrity, and the Unique Key keeps all values different. Used together, they make a database organised, consistent and safe.

04 QUESTION FOUR · DBMS VS RDBMS

Differentiate between DBMS and RDBMS.

4 A4 PAGES
SQL JOIN EXAMPLE ON PAGE 4

WHAT IS A DBMS?

A **DBMS (Database Management System)** is software that stores, retrieves and manages data. A DBMS may keep its data in files, in a hierarchy or in a network — it does not have to follow the relational model. Its job is to organise the data, protect it and provide backup and recovery. In a DBMS there is no automatic relationship between tables and no true JOIN operation, so related information must be looked up and combined by the program itself. Examples of DBMS include plain file systems, XML databases and NoSQL databases such as MongoDB, as well as simple tools such as Microsoft Access when they are used without keys. A DBMS is well suited to small and simple applications where the amount of data is limited and complex relationships between tables are not needed.

Characteristics of a DBMS

- Data is stored in files, hierarchies or networks.
- There is no strong relationship between tables.
- Normalization is not required.
- There is no concept of primary or foreign keys.
- The JOIN operation is not available.
- ACID properties are not fully supported.
- It suits small, single-user or simple applications.

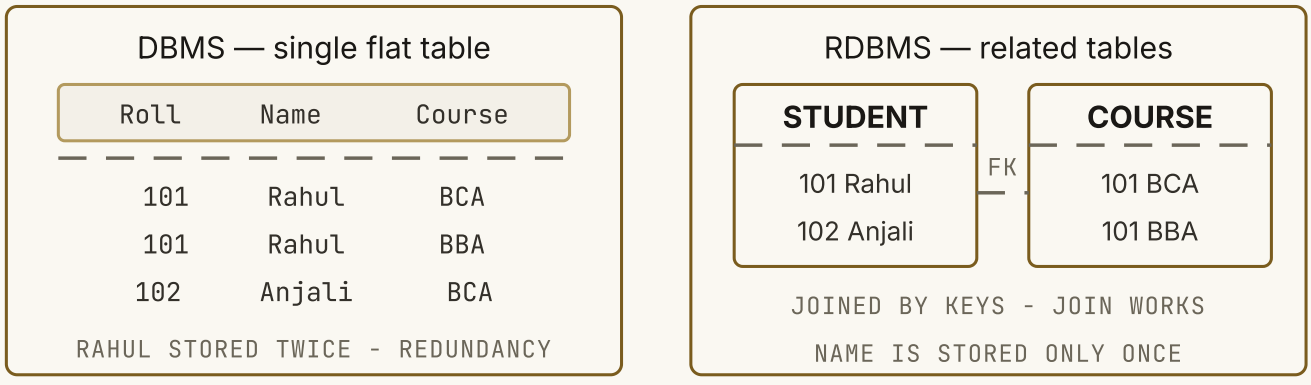


FIG. 1 — A DBMS KEEPS ONE FLAT TABLE AND REPEATS DATA; AN RDBMS KEEPS SMALL RELATED TABLES, JOINS THEM WITH KEYS, AND STORES EACH FACT ONLY ONCE.

WHAT IS AN RDBMS?

An **RDBMS (Relational Database Management System)** is a special form of DBMS in which data is stored in **relations** — structured tables of rows and columns — connected to one another through common keys. It is based on the **relational model** proposed by **E. F. Codd** and queried using **SQL (Structured Query Language)**. Its greatest feature is the **JOIN** operation, which combines related tables so information spread across many tables is retrieved as one result.

Examples include **Oracle, MySQL, Microsoft SQL Server, PostgreSQL** and **SQLite**. An RDBMS is best for large, complex applications where there is a great deal of data and where the relationships between tables are essential. It fully supports the **ACID properties** — Atomicity, Consistency, Isolation and Durability — because of which transactions remain safe even if the system fails midway through one.

Characteristics of an RDBMS

- Data is stored in structured tables of rows and columns.
- There are strong relationships between tables through foreign keys.
- Normalization is required to remove redundancy.
- Primary keys, foreign keys and composite keys are supported.
- The JOIN operation is available in its full form.
- ACID properties are fully supported, so transactions are safe.
- It supports many users and large volumes of data at once.

Beyond the table structure, the greatest strength of an RDBMS is that it protects the data even when many users are working on it at the same moment. Suppose one clerk is transferring money from one account to another while another is preparing a report on the same accounts. The RDBMS uses its **transaction** and **locking** rules to make sure the report never sees a half-finished transfer. This ability to keep data correct under simultaneous use is called **concurrency control**, and it is why banks and large organizations insist on an RDBMS rather than a simple DBMS.

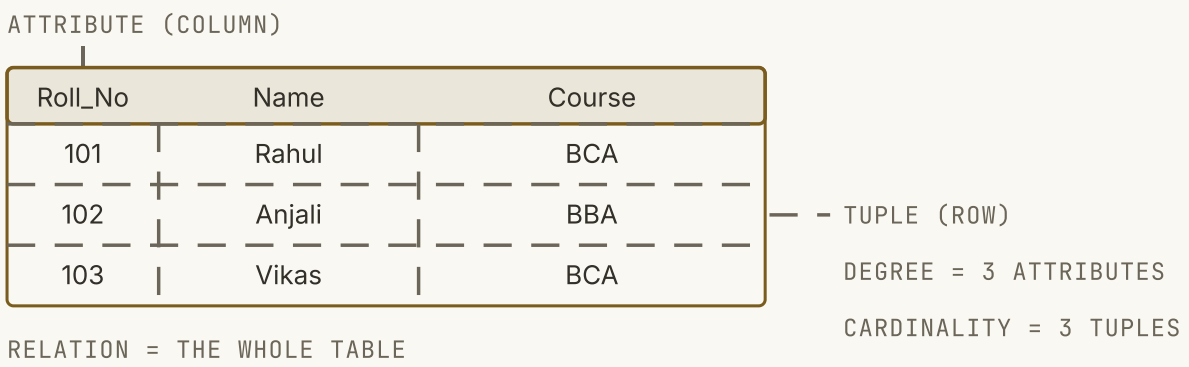


FIG. 2 — THE WORDS AN RDBMS USES FOR THE PARTS OF A TABLE. A RELATION IS THE WHOLE TABLE, A TUPLE IS ONE ROW, AN ATTRIBUTE IS ONE COLUMN; THE DEGREE COUNTS THE ATTRIBUTES AND THE CARDINALITY COUNTS THE TUPLES. EVERY TABLE IN A RELATIONAL DATABASE IS BUILT FROM EXACTLY THESE PARTS.

DIFFERENCE BETWEEN DBMS AND RDBMS

BASIS	DBMS	RDBMS
Data storage	In files, hierarchies or networks	In structured tables (relations)
Structure	Unstructured or semi-structured	Structured — rows and columns
Normalization	Not required	Required to remove redundancy
Keys	No primary or foreign keys	Primary, foreign and composite keys
Relationships	No relationship between tables	Tables related through foreign keys
JOIN operation	Not available	Inner, left, right and full joins
ACID properties	Not fully supported	Fully supported
Query language	Limited query processing	SQL — a powerful standard
Examples	File system, XML, MongoDB	Oracle, MySQL, SQL Server
Best for	Small, simple applications	Large, multi-user applications

The most important difference is the **JOIN**. In a DBMS, if two pieces of related information are kept in different places, the program must fetch both and combine them itself; in an RDBMS the database does this work efficiently with a single JOIN command. On the next page we see this operation in action.

It should also be understood that an RDBMS is not merely a “better DBMS”; it is a DBMS that follows one particular model — the relational model. Every RDBMS is therefore a DBMS, but every DBMS need not be an RDBMS. This is why the two are compared rather than placed in opposition: the RDBMS adds a structure, a query language and a set of guarantees (the keys and the ACID properties) on top of the basic idea of managing data. When the data is small and simple, that extra structure is unnecessary work; when the data is large, that same structure is exactly what keeps the system correct.

SQL JOIN EXAMPLE (RDBMS)

TWO RELATED TABLES JOINED TOGETHER — AN OPERATION THAT IS POSSIBLE ONLY IN AN RDBMS. COPY THIS PROGRAM ON PAGE 4.

DBMS_VS_RDBMS.SQL

```
-- two related tables : a JOIN made possible by the foreign key
CREATE TABLE course (
    course_code VARCHAR2(10) PRIMARY KEY,
    course_name VARCHAR2(40),
    fee         NUMBER(8,2)
);
CREATE TABLE student (
    roll_no     NUMBER(5) PRIMARY KEY,
    name        VARCHAR2(40),
    course_code VARCHAR2(10) REFERENCES course(course_code)
);
INSERT INTO course VALUES ('BCA101','Java Programming',5000);
INSERT INTO course VALUES ('BCA102','Oracle SQL',      4500);
INSERT INTO student VALUES (101,'Rahul Kumar', 'BCA101');
INSERT INTO student VALUES (102,'Anjali Singh','BCA102');
INSERT INTO student VALUES (103,'Vikash Yadav','BCA101');

-- INNER JOIN : students shown together with their course
SELECT s.roll_no, s.name, c.course_name, c.fee
FROM   student s INNER JOIN course c
ON     s.course_code = c.course_code
ORDER  BY s.roll_no;
```

Output of the Query

OUTPUT

```
ROLL_NO NAME          COURSE_NAME      FEE
------- ------------- ---------------- -----
101     Rahul Kumar   Java Programming 5000
102     Anjali Singh  Oracle SQL       4500
103     Vikash Yadav  Java Programming 5000
```

CONCLUSION

In the JOIN above, an RDBMS combined two related tables and produced a single result. This operation is not possible in a DBMS, which is exactly why an RDBMS is used for large and complex applications while a DBMS is enough for small, simple ones.